

Multi-Level Feedback Queue on xv6-RISC-V

Week 2: MLFQ Scheduler Core

Aadesh Panjwani and M. Rayyan Khan

1 Overview

In Week 2, the xv6 scheduler was modified to implement a Multi-Level Feedback Queue (MLFQ) system, replacing the original simple round-robin scheduler. The goals were:

- Introduce multiple priority queues to support CPU-bound vs I/O-bound differentiation.
- Implement per-process bookkeeping for time slices, queue levels, and runtime metrics.
- Maintain fairness while prioritizing interactive tasks.

2 Scheduler Implementation

2.1 Queue Structures and Locking

Each queue is guarded by a spinlock for safe concurrent access:

```
1 struct spinlock mlfq_lock;
2
3 struct procqueue {
4     struct proc *head;
5     struct proc *tail;
6 };
7
8 struct mlfq_state {
9     struct procqueue queues[MLFQ_LEVELS];
10 };
```

2.2 Process Enqueue / Dequeue

Processes are inserted either at the head or tail of a queue depending on context:

```
1 static void mlfq_make_runnable(struct proc *p,
2                               enum mlfq_enqueue_pos pos,
3                               int reset_budget) {
4     if(p->state != RUNNABLE) return;
5     if(reset_budget) mlfq_reset_budget(p);
6     mlfq_enqueue_proc(p, pos);
7 }
```

2.3 Scheduler Tick

Timer interrupts now charge running processes and handle demotion:

```
1 int scheduler_tick(void) {
2     struct proc *p = myproc();
3     if(p == 0 || p->state != RUNNING) return 0;
4
5     p->total_runtime++;
6     p->queue_runtime[p->queue_level]++;
7     if(p->time_slice_budget > 0) p->time_slice_budget--;
8
9     if(p->time_slice_budget <= 0) {
10        if(p->queue_level < MLFQ_LEVELS - 1)
11            p->queue_level++;
12        mlfq_reset_budget(p);
13        return 1;
14    }
15    return 0;
16 }
```

2.4 Scheduler Loop

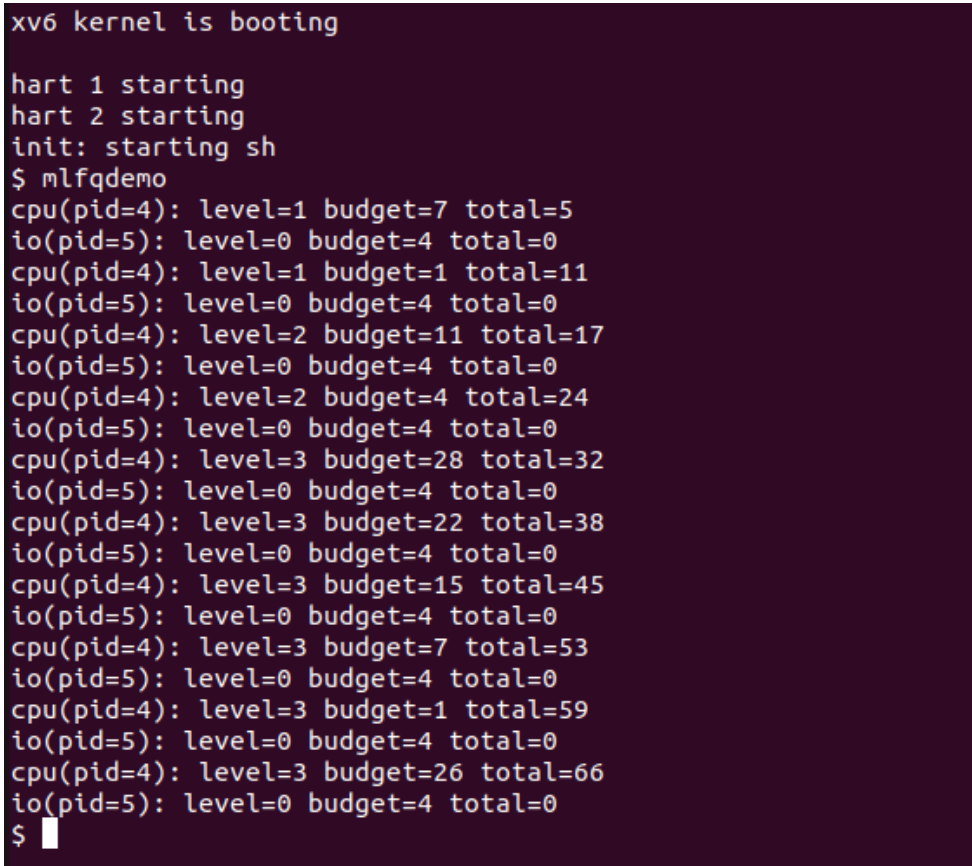
```
1 void scheduler(void) {
2     struct cpu *c = mycpu();
3     c->proc = 0;
4
5     for(;;){
6         intr_on();
7         intr_off();
8
9         struct proc *p = mlfq_pick_next();
10        if(p == 0){
11            asm volatile("wfi");
12            continue;
13        }
14
15        p->state = RUNNING;
16        c->proc = p;
17        swtch(&c->context, &p->context);
18
19        c->proc = 0;
20        if(p->state == RUNNABLE)
21            mlfq_make_runnable(p, ENQUEUE_TAIL, 0);
22        release(&p->lock);
23    }
24 }
```

3 Queue & Wakeup Policy

- Sleeping tasks are inserted at the head of the queue when woken, ensuring low-latency response for I/O-bound tasks.
- Yielding processes keep their current priority but have their budget reset, preserving fairness.
- Newly created processes always start at the top queue with a full time slice.

4 Demo Results

The included user program `mlfqdemo` spawns a CPU-bound and an I/O-bound process. The parent samples their runtime statistics over time.



```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ mlfqdemo
cpu(pid=4): level=1 budget=7 total=5
io(pid=5): level=0 budget=4 total=0
cpu(pid=4): level=1 budget=1 total=11
io(pid=5): level=0 budget=4 total=0
cpu(pid=4): level=2 budget=11 total=17
io(pid=5): level=0 budget=4 total=0
cpu(pid=4): level=2 budget=4 total=24
io(pid=5): level=0 budget=4 total=0
cpu(pid=4): level=3 budget=28 total=32
io(pid=5): level=0 budget=4 total=0
cpu(pid=4): level=3 budget=22 total=38
io(pid=5): level=0 budget=4 total=0
cpu(pid=4): level=3 budget=15 total=45
io(pid=5): level=0 budget=4 total=0
cpu(pid=4): level=3 budget=7 total=53
io(pid=5): level=0 budget=4 total=0
cpu(pid=4): level=3 budget=1 total=59
io(pid=5): level=0 budget=4 total=0
cpu(pid=4): level=3 budget=26 total=66
io(pid=5): level=0 budget=4 total=0
$
```

Figure 1: CPU-bound process moving to lower priority queues over time. I/O-bound process stays in the higher priority queue.

5 Design Update

- Per-process bookkeeping fields added: `queue_level`, `time_slice_budget`, `queued`, `queue_runtime`.
- Scheduler loop simplified to pick the next runnable process from highest non-empty queue.
- `scheduler_tick` separates time slice tracking from CPU yield decisions.

6 Next Steps

- Implement periodic priority boosting.
- Audit starvation scenarios and validate fairness across workloads.
- Extend ‘mlfqdemo’ for automated testing of multiple simultaneous processes.